

OData Services

Adrian Streitz, Johannes Rank, Borys Levkovskyi



Prerequisites

- Fundamental programming skills
- Successful completion of the chapter *Introduction to SAP Application Development*
- Recommended: Completion of the chapters *HTML5, CSS & JavaScript* and *SAPUI5*
- Completion of the chapter *ABAP Objects* is helpful

Agenda

- I. Fundamentals**
- II. Using the OData protocol in SAP**
- III. Creating and implementing OData services**
- IV. Consuming OData services within SAPUI5**

OData

- OData = Open Data Protocol
- HTTP-based standard for data services and data transfers
- HTTP requests are transferred using *Asynchronous JavaScript and XML* (AJAX)
- SAP Gateway allows to access SAP backend services using the OData protocol
- HTTP responses contains data from the SAP system in OData format
- OData services follow the principle of RESTful webservice

I. Fundamentals

Communication with the SAP backend

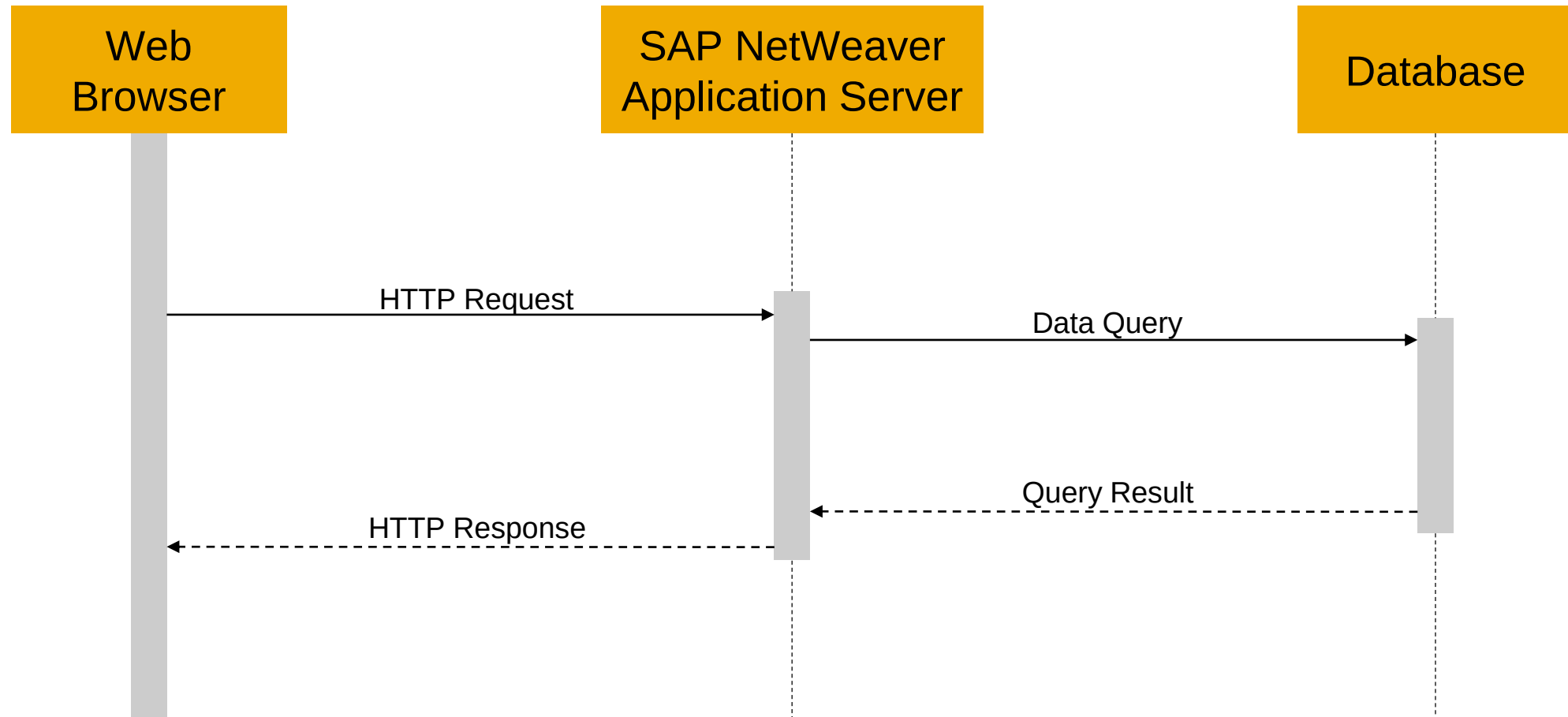
The screenshot shows the 'Create Purchase Order' app in a web browser. The browser address bar displays a URL: `https://i88lp1.informatik.tu-muenchen.de:8400/fiori?sap-client=900&sap-language=EN&sap-sec_session_created=X#PurchaseOrder-create&/createPC`. The app interface includes a top bar with the SAP logo and a search icon. Below the bar, the 'General Data' section contains input fields for Supplier, Company Code, Currency, Purchasing Group, and Purchasing Organization, along with a 'Total PO Value' of 0.00. The 'Items' section features a table with columns for Item, Material Group, Plant, Quantity, Unit of Measure, Net Price, Per, Price Unit, and Item Value. The 'Delivery and Invoice' section includes fields for Payment Terms, Invoicing Party, Incoterm, Incoterm Location, and Exchange Rate.

AJAX

JSON, XML, OData

SAP
NetWeaver
AS ABAP

Asynchronous JavaScript and XML (AJAX)



REST

- REST is a form of a service-oriented architecture
- Describes an architectural standard for client-server communication, based on HTTP
- Important principles:
 - **Addressability**: content is represented by a Universal Resource Identifier (URI)
 - **Different representations**: a service can have different presentation formats (OData, XML, JSON, ...)
 - **Stateless**: each REST message contains all necessary information for the client to understand it
 - **Operations**: services provide operations to access or change resources and information
 - **Hypermedia**: linking of information via hyperlinks

URIs

- URI = Unified Resource Identifier
- Every Resource of a REST WebService is represented by an URI
- A Resource is an Entity like Customer, Product, Material, ...
- A client uses the HTTP methods to work with the resources (GET, PUT, POST, DELETE)

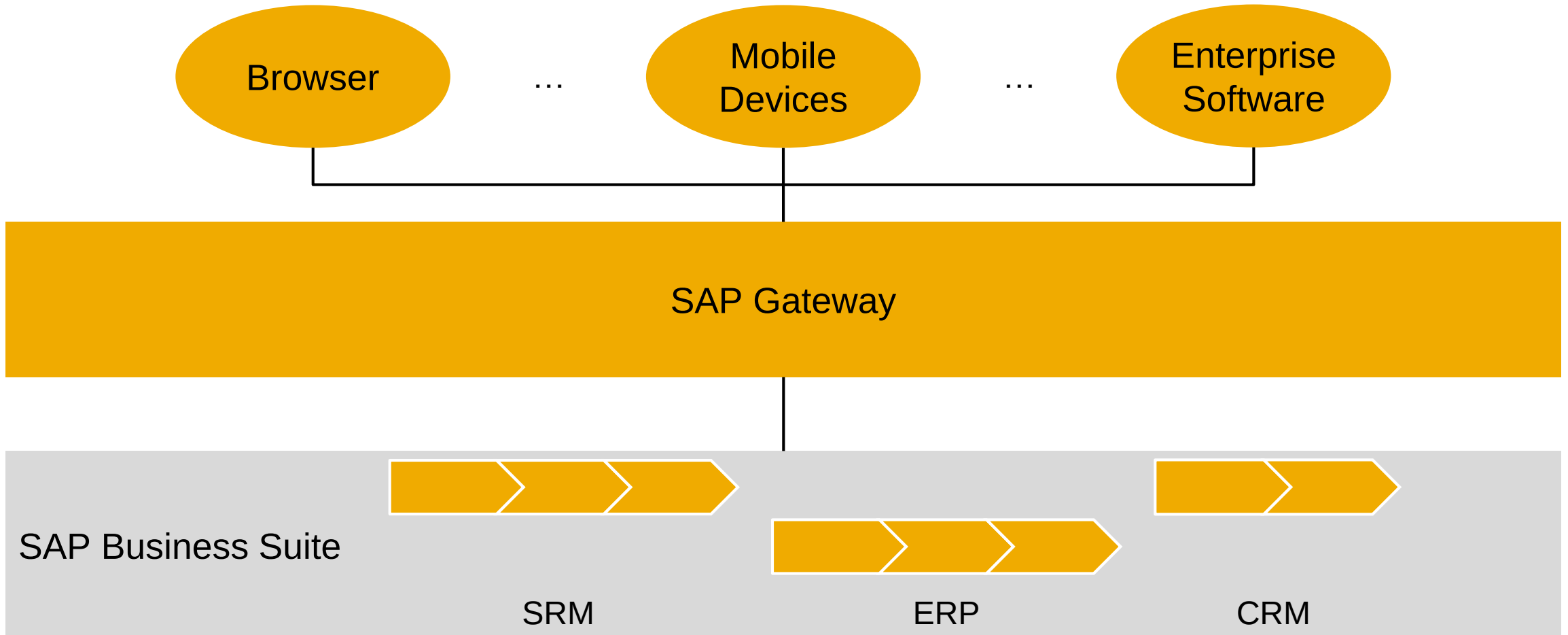
▼ Service Implementation

▼ FlightSet

-  Create
-  Delete
-  GetEntity (Read)
-  GetEntitySet (Query)
-  Update

II. Using the OData protocol in SAP

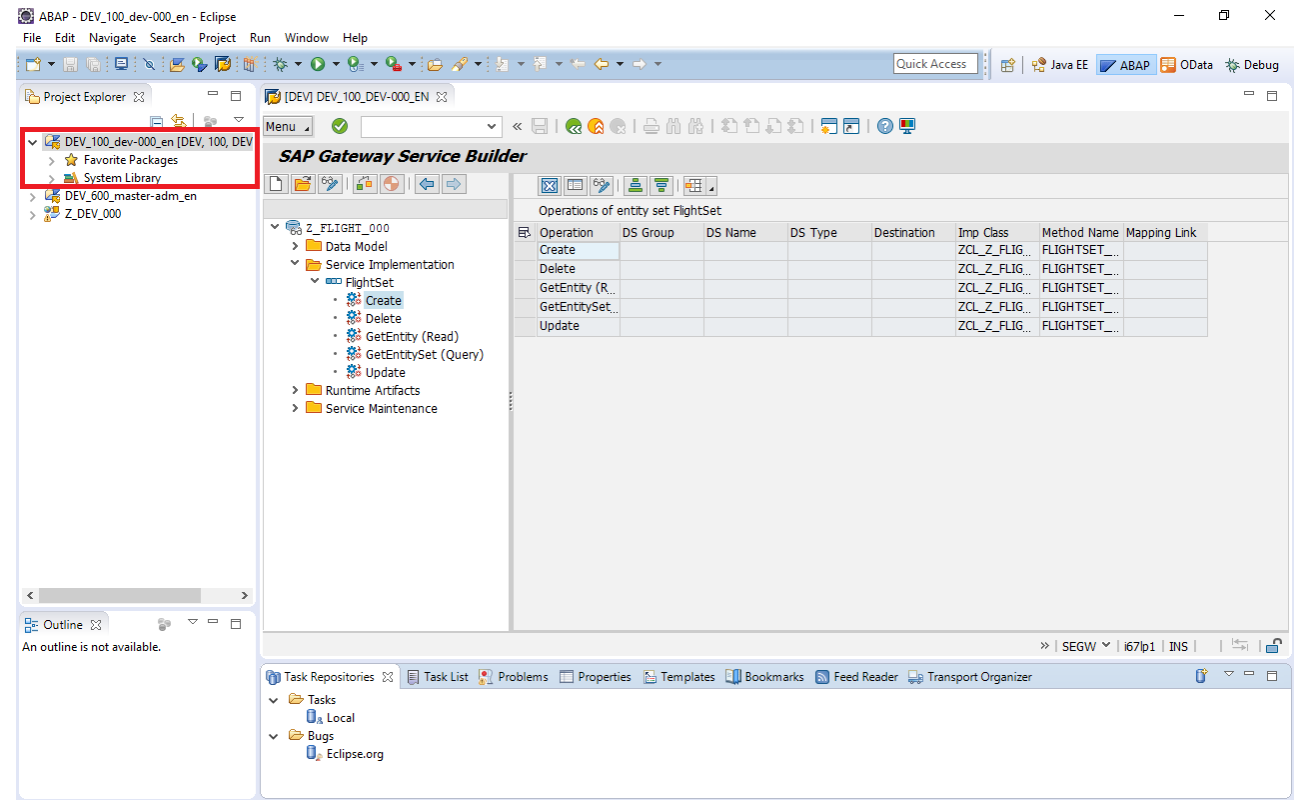
SAP Gateway



III. Creating and implementing OData services

Step 1: Create a new ABAP project in Eclipse

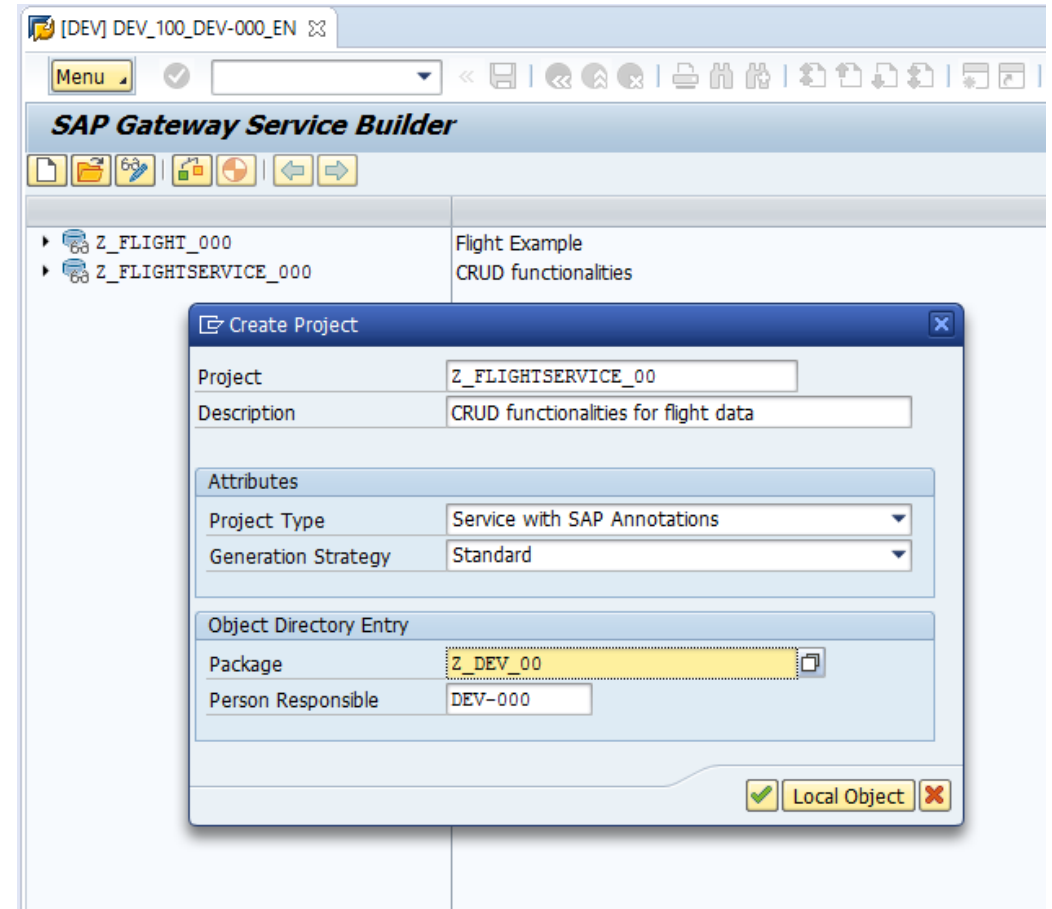
- The ABAP project is needed to access the integrated SAP GUI with Eclipse
- OData services have to be implemented within an ABAP package
- The integrated SAP GUI in Eclipse serves to implement, activate, and test the OData service
- Eclipse can be used to implement the service logic in ABAP



III. Creating and implementing OData services

Step 2: Create a new project in the Gateway Service Builder

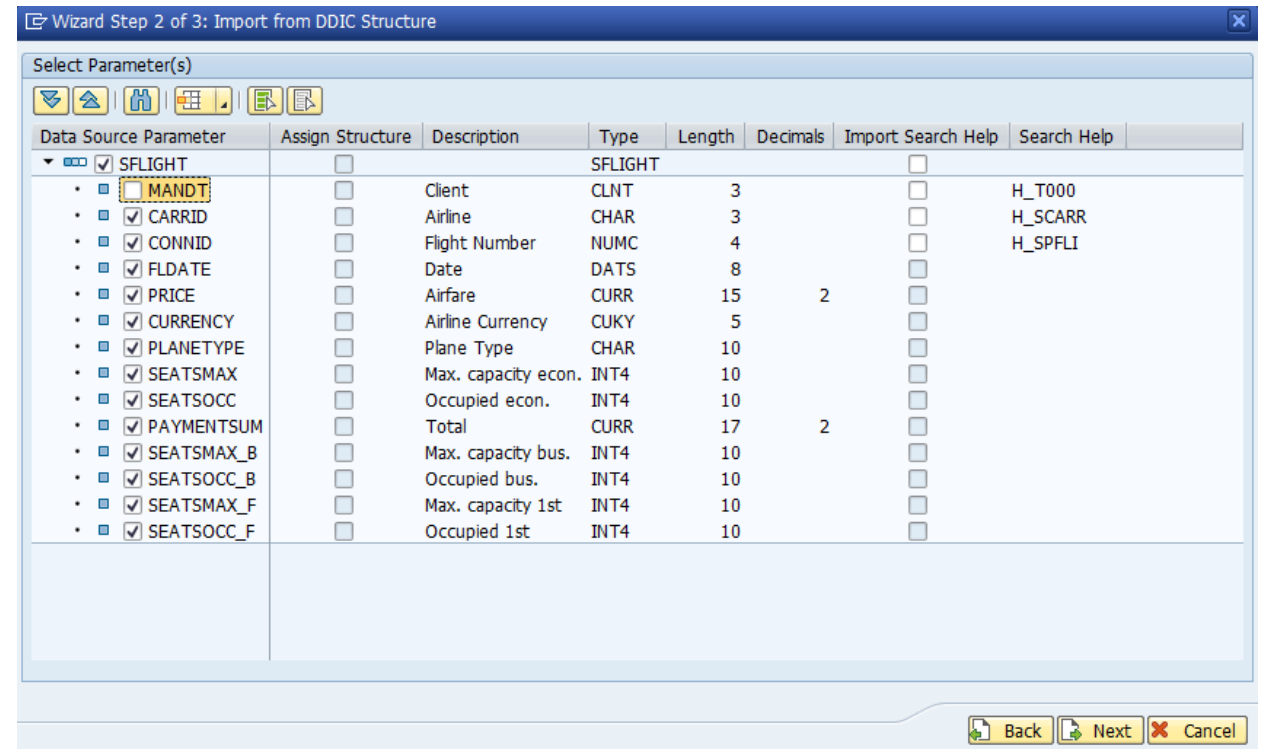
- The project within the Gateway Service Builder serves as container for a group of services
- The project generates four directories:
 - **Data Model:** Binding data models
 - **Service Implementation:** ABAP implementation of service logic
 - **Runtime artefacts:** Service documents
 - **Service Maintenance:** Managing Gateway connection



III. Creating and implementing OData services

Step 3: Create data model

- There are different sources to create data models (e.g. using flat files)
- The most common way is using existing database objects in SAP (DDIC structures)
- You have to select the fields you want to import from a DDIC structure
- At least one key element has to be defined out of the field collection to serve as a unique identifier for a data entry



III. Creating and implementing OData services

Step 4: Generate service elements

- Different service elements are automatically created by the Gateway Service Builder
- Four main classes are created:
 - **..._MPC**: Model definitions
 - **..._MPC_EXT**: inherits from the MPC-class, allows to overwrite service models
 - **..._DPC**: Data provider class, contains CRUD-functionalities
 - **..._DPC_EXT**: inherits from DPC class, allows to overwrite CRUD-functionalities

The screenshot shows the 'Model and Service Definition' dialog box with three sections: Model Provider Class, Data Provider Class, and Service Registration.

Model Provider Class	
Class Name	ZCL_Z_FLIGHTSERVICE_00_MPC_EXT
Base Class Name	ZCL_Z_FLIGHTSERVICE_00_MPC

Data Provider Class	
☒ Generate Classes	
Class Name	ZCL_Z_FLIGHTSERVICE_00_DPC_EXT
Base Class Name	ZCL_Z_FLIGHTSERVICE_00_DPC

Service Registration	
Technical Model Name	Z_FLIGHTSERVICE_00_MDL
Model Version	1
Technical Service Name	Z_FLIGHTSERVICE_00_SRV
Service Version	1

At the bottom right of the dialog, there are two icons: a green checkmark and a red X.

III. Creating and implementing OData services

Model Provider Class (MPC)

Model Provider Class:

- Defines the entity data model of a service
- Is generated based on the odata model by the Service Builder
- (Has to be/ is) regenerated every time the model definition is changed
- Model definitions, **shouldn't be changed**
- **_Extension:** Implementation class, inherits from the MPC-class, used for manual changes -> **can be changed** -> append new properties to corresponding type -> add new property to existing entity

III. Creating and implementing OData services

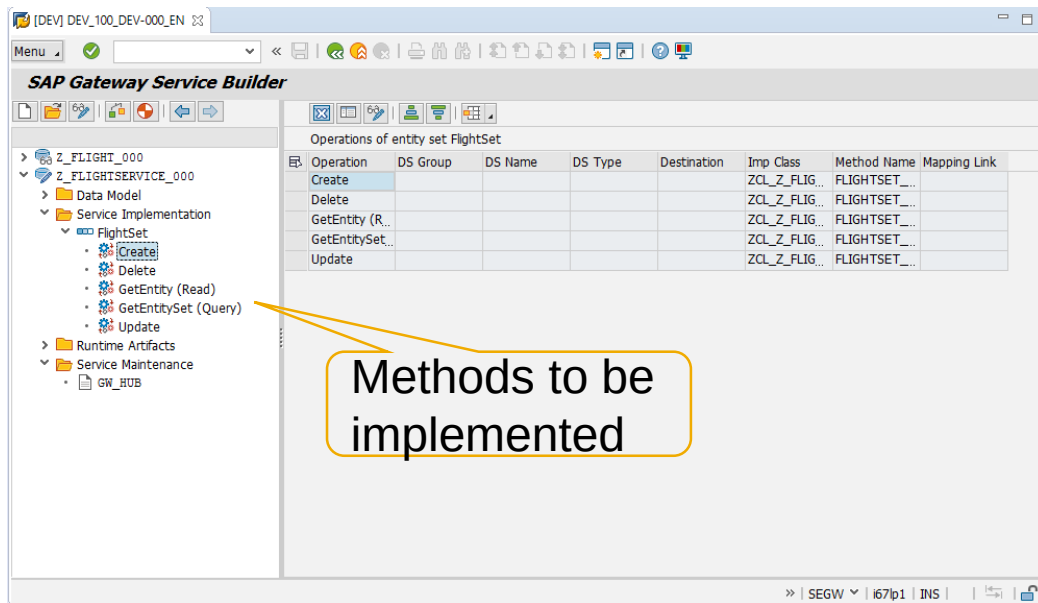
Model Provider Class (MPC)

- Different service elements are automatically created by the Gateway Service Builder
- Four main classes are created:
 - **Model Provider Class:** Model definitions, Define()-one for each odata artifact (entity types, complex types ...) and GET_LAST_MODIFIED(). Use define() to create entity, properties etc. -> **shouldn't be changed** -Provides runtime representation of your model definition
 - **_Extension:** Implementation class, inherits from the MPC-class, used for manual changes -> **can be changed** -> append new properties to corresponding type
 - **Data Provider Class:** Data provider class, implements: Create, Read, Update, Delete (CRUD) and Query -> runtime representation of service implementation
 - **_Extension:** Implementation class, inherits from DPC class, allows to overwrite/ redefine methods for the DPC.... The one that is executed in odata service ->is executed in odata service

III. Creating and implementing OData services

Step 5: Implement services

- The CRUD functionalities have to be implemented in ABAP in order to process data operations
- The coding can be done in Eclipse



```
CLASS zcl_z_flightservice_00_dpc_ext IMPLEMENTATION.  
  
METHOD flightset_create_entity.  
  
DATA: ls_request_input_data TYPE zcl_z_flightservice_00_mpc=>ts_flight,  
      ls_flight TYPE sflight.  
  
*Read request  
io_data_provider->read_entry_data( IMPORTING es_data = ls_request_input_data ).  
  
* Assign flight data values to fields  
ls_flight-carrid = ls_request_input_data-carrid.  
ls_flight-connid = ls_request_input_data-connid.  
ls_flight-currency = ls_request_input_data-currency.  
ls_flight-fldate = ls_request_input_data-fldate.  
ls_flight-paymentsum = ls_request_input_data-paymentsum.  
ls_flight-planetype = ls_request_input_data-planetype.  
ls_flight-price = ls_request_input_data-price.  
ls_flight-seatsmax = ls_request_input_data-seatsmax.  
ls_flight-seatsmax_b = ls_request_input_data-seatsmax_b.  
ls_flight-seatsmax_f = ls_request_input_data-seatsmax_f.  
ls_flight-seatsocc = ls_request_input_data-seatsocc.  
ls_flight-seatsocc_b = ls_request_input_data-seatsocc_b.  
ls_flight-seatsocc_f = ls_request_input_data-seatsocc_f.  
  
*Insert data into table SFLIGHT  
INSERT sflight FROM ls_flight.  
  
ENDMETHOD.  
ENDCLASS.
```

Read HTTP request

Assign values to data fields

Insert statement

Now you know the fundamentals of OData services.

To consolidate your knowledge, you can do tasks 1 and 2 of the OData exercise.

III. Creating and implementing OData services

Step 6: Activate services

- Call transaction `/n/IWFND/MAINT_SERVICE`
- Add your newly created service to the service catalog

The 'Add Service' dialog box is shown with the following fields and values:

- Service**
 - Technical Service Name: `Z_FLIGHTSERVICE_000_SRV`
 - Service Version: `1`
 - Description: `CRUD functionalities`
 - External Service Name: `Z_FLIGHTSERVICE_000_SRV`
 - Namespace: (empty)
 - External Mapping ID: (empty)
 - External Data Source Type: `C`
- Model**
 - Technical Model Name: `Z_FLIGHTSERVICE_000_MDL`
 - Model Version: `1`
- Creation Information**
 - Package Assignment: `Z_DEV_000`
 - Local Object: (selected)
- ICF Node**
 - ☒ Standard Mode ☐ None
 - ☒ Set Current Client as Default Client in ICF Node
- OAuth enablement**
 - ☐ Enable OAuth for Service

The 'Activate and Maintain Services' transaction shows the 'Service Catalog' table with the following data:

Type	Technical Service Name	Service Description	External Service Name	Namespace	OAuth	Soft State	Status	Is SAP Service
BEP	/IWFND/SG_MED_CATALOG	1 Catalog Service	CATALOGSERVICE	/IWFND/	☐	Not Supported		
BEP	/IWFND/SG_MED_CATALOG	2 Catalog Service Version 2	CATALOGSERVICE	/IWFND/	☐			
BEP	DAAG_MINGGRP	1 Data Aging Manage Groups	DAAG_MINGGRP		☒			
BEP	DAAG_MONOBJ	1 Data Aging Monitor Objects	DAAG_MONOBJ		☒			
BEP	EPM_LANES_DEMO_SRV	1 EPM: Gateway Service for Lanes Page Demo	EPM_LANES_DEMO_SRV		☒			
BEP	EPM_OIA_APPS_GW_SERVICE_SRV	1 CL_EPM_OIA_APPS_GW_SERV_DPC_EXT	EPM_OIA_APPS_GW_SERVICE_SRV		☒			
BEP	EPM_OIA_BPFACTSHEET_SRV	1 EPM OIA: Open Item Analysis Dashboard - BP Fact...	EPM_OIA_BPFACTSHEET_SRV		☒			
BEP	EPM_OIA_BROPENINV_SRV	1 EPM OIA:Open Item Analysis Dashboard - BP Ope...	EPM_OIA_BROPENINV_SRV		☒			
BEP	EPM_OIA_CTRYSALESTREND_SRV	1 EPM OIA: Open Item Analysis Dashboard - Countr...	EPM_OIA_CTRYSALESTREND_SRV		☒			
BEP	EPM_OIA_DFG_GW_SERVICE_SRV	1 CL_EPM_OIA_DFG_GW_SERV_DPC_EXT	EPM_OIA_DFG_GW_SERVICE_SRV		☐			
BEP	EPM_OIA_FIORI_GW_SERVICE_SRV	1	EPM_OIA_FIORI_GW_SERVICE_SRV		☐			
BEP	EPM_OIA_FIORI_SRV	1 EPM OIA: Open Item Analysis Dashboard - Launch...	EPM_OIA_FIORI_SRV		☒			
BEP	EPM_OIA_GW_SALES_HISTORY_SRV	1 CL_EPM_OIA_GW_SALES_HI_DPC_EXT	EPM_OIA_GW_SALES_HISTORY_SRV		☒			
BEP	EPM_OIA_L_1_SRV_01	1 SAP Lumra for OIA Dashboard	EPM_OIA_L_1_SRV_01		☒			
BEP	EPM_OIA_L_1_SRV_02	1 SAP Lumra for OIA Dashboard	EPM_OIA_L_1_SRV_02		☒			
BEP	EPM_OIA_OPENINV_SRV	1 EPM OIA:Open Item Analysis Dashboard - Open In	EPM_OIA_OPENINV_SRV		☒			

Below the table, the 'ICF Nodes' section shows:

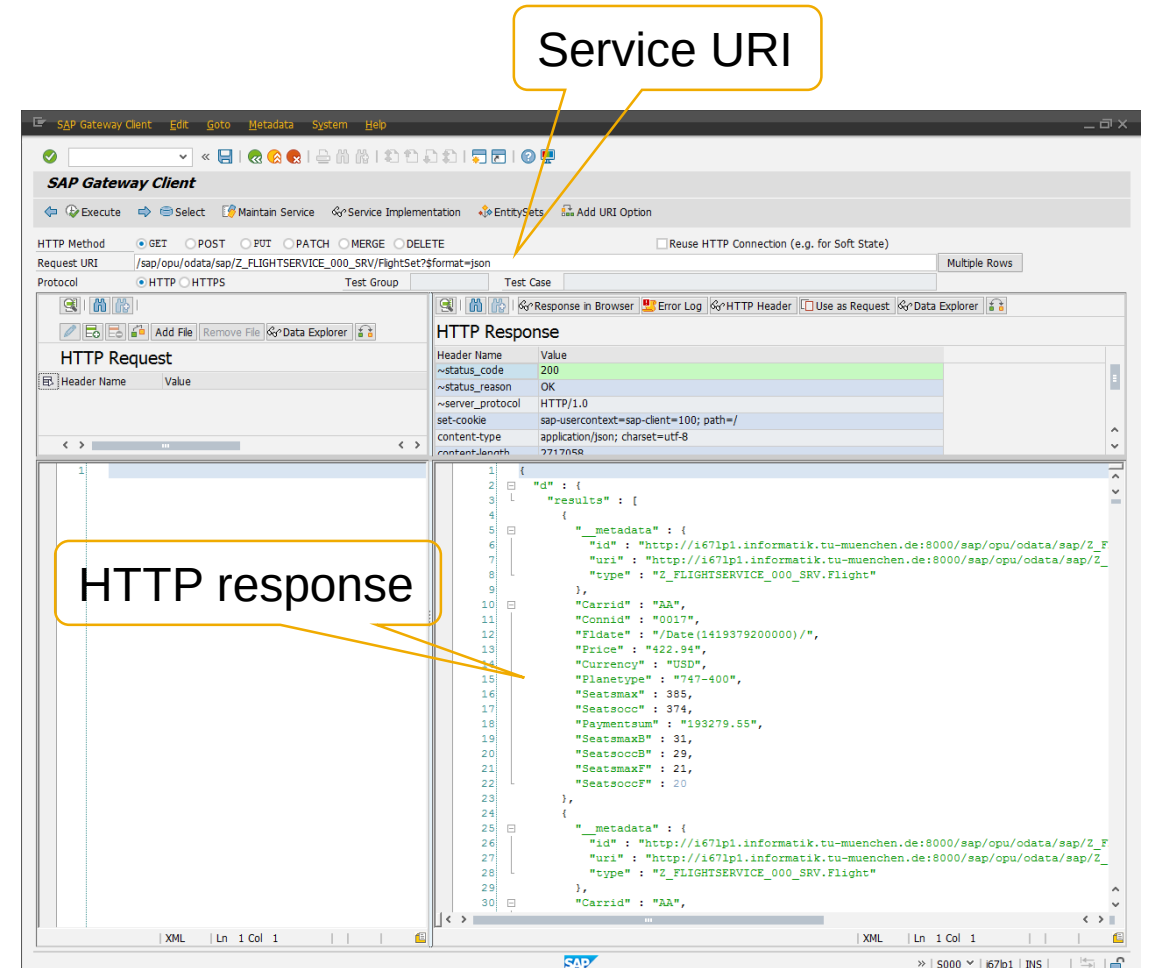
Status	ICF Node	Session Time-out	Soft State	Description
☒	ODATA	00:00:00		Standard Mode

The 'System Aliases' section is also visible, showing a table with columns: SAP System Alias, Description, Default, Metadata, User Role.

III. Creating and implementing OData services

Step 7: Test services

- Use transaction **/n/IWFND/MAINT_SERVICE**
- Select your service and call the SAP Gateway Client
- With the Gateway Client, you can test services by calling the respective URI with an HTTP request
- You can directly see the HTTP response in the requested format



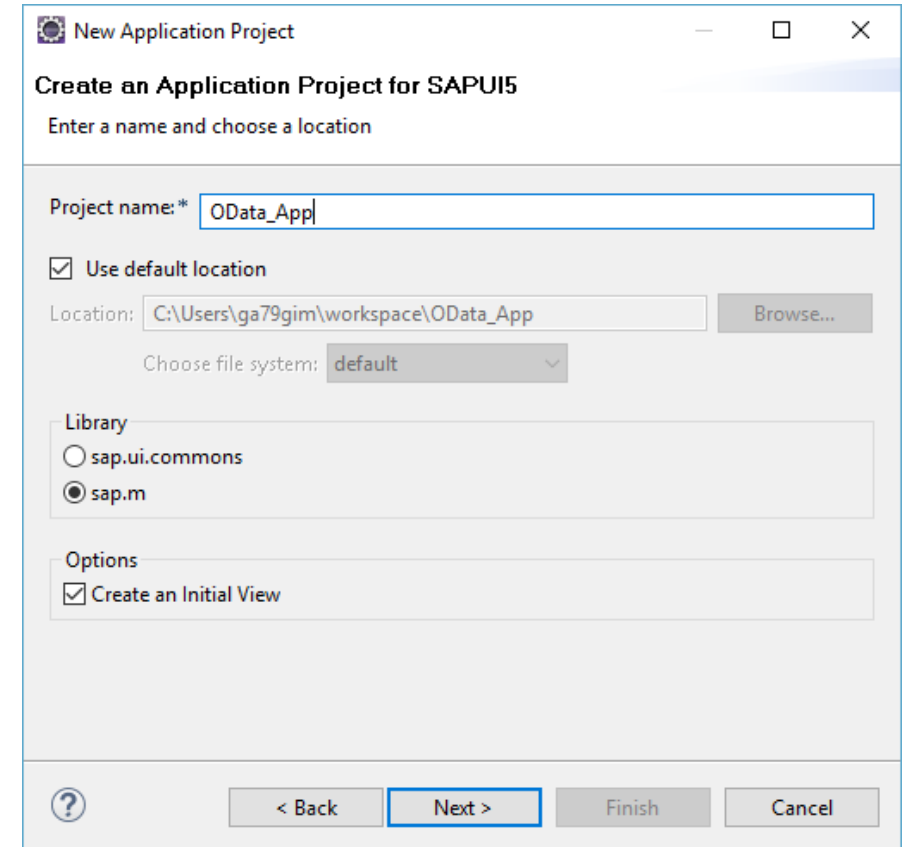
Now you know the complete process of creating and implementing OData services.

To consolidate your knowledge, you can do task 3 of the OData exercise.

IV. Consuming OData services within SAPUI5

Step 1: Create a new SAPUI5 application project

- In the Project Explorer, choose *New -> Other -> SAPUI5 Application Development -> Application Project*
- It is recommended to create an initial view
- You can choose between different formats for your initial view:
 - JavaScript
 - XML
 - JSON
 - HTML



IV. Consuming OData services within SAPUI5

Step 2: Implement the service call

```
<core:View xmlns:core="sap.ui.core" xmlns:mvc="sap.ui.core.mvc"
  xmlns="sap.m" controllerName="odata_app.main" xmlns:html="http://www.w3.org/1999/xhtml">
  <Page title="Flight List">
    <content>
      <Table id="flightsTable" items="{ path: '/FlightSet' }" growing="true"
        growingScrollToLoad="true">
        <columns>
          <Column>
            <Text text="ConnID" />
          </Column>
          <Column>
            <Text text="CarrID" />
          </Column>
        </columns>
        <items>
          <ColumnListItem>
            <cells>
              <Text text="{connid}" />
              <Text text="{carrid}" />
            </cells>
          </ColumnListItem>
        </items>
      </Table>
    </content>
  </Page>
</core:View>
```

Definition of table columns

Path to FlightSet

Definition of table data rows

Binding data model to view

Service URI

```
onInit : function() {
  alert("Hello");

  var serviceUrl = getServiceUrl("/sap/opu/odata/sap/Z_FLIGHTSERVICE_000_SRV/");
  var oModel = new sap.ui.model.odata.v2.ODataModel(
    serviceUrl);

  this.getView().setModel(oModel);

},
```

Step 2: Create operation

```
// create data set
var oNewUser = {
    Userid      : 'NEW_USER',
    Fname       : 'New',
    Lname       : 'User',
    Function     : 'tbd',
    Department   : 'tbd',
    Phone       : '666',
    Mail        : 'no-reply@myself.com',
};

var oModel = this.getView().getModel();
oModel.create(
    '/UserSet',
    oNewUser,
    {
        success: function (oData, oResponse){
            sap.m.MessageToast.show('User created');
        };
        error: function (oError){
            sap.m.MessageBox.error('Error by creation');
        }
    }
);
```

Callback functions

Step 2: Update operation

```
// data, which shall be changed
var oChangeUser = {
    Function      : 'New Function',
    Department    : 'New Department',
};

var oModel = this.getView().getModel();
oModel.update(
    '/UserSet('NEW_USER')',
    oChangeUser,
    {
        success: function (oData, oResponse){
            sap.m.MessageToast.show('User updated');
        },
        error: function (oError){
            sap.m.MessageBox.error('Error by updating');
        },
        merge: true
    }
);
```

Callback functions

Is the whole data provided or only a part of it?

Step 2: Delete operation

```
onDeleteUser: function (oEvent){
    var oModel = this.getView().getModel();

    oModel.remove(
        '/UserSet('NEW_USER')',
        {
            success: function (oData, oResponse){
                sap.m.MessageToast.show('User removed');
            },
            error: function (oError){
                sap.m.MessageBox.error('Error by removing');
            }
        }
    );
}
```

Callback functions

IV. Consuming OData services within SAPUI5

Step 3: Deploy the application in the SAP backend

- In the Project Explorer, choose *Team -> Share Project*
- Choose the *SAPUI5 ABAP Repository* as Repository object
- Connect to the SAP system to which you want to deploy the application
- Create a new BSP application as container for your SAPUI5 implementation
- You can repeatedly submit implementation changes to the SAP backend

Share Project

SAPUI5 Repository

Specify a BSP application

☐ Select a BSP Application:

Name	Description
/UI2/LAUNCHPAGE	Launchpage for Tile CHiPs
/UI2/TILECHIPS	CHiPs that can be used in Launchpage
/UI2/USHELL	Unified Shell
/UI2/USHELL_GWA	Generic Wrapper Application
/UI2/USHELL_ME	UShell Me Profile
/UI5/SIMPLETEST	Hello World
AOC	ABAP Online Community - SAPUI5 Application
ARSRVC_LPD_C	Actions Reusable CHIP
ARSRVC_SM	Social Media Productivity Services
ARSRVC_SPB_ADMIN	Suite Page Builder - Administration
ARSRVC_SUITE_PB	Suite Page Builder
ARSRVC_UPB_ADMIN	Unified Shell Page Builder Administration Page
AR_SRVC_NEWS	News Tile
BPCWEBCLIENT	BPC universal web client

BSP Application:

☒ Create a new BSP Application

Name *

Description *

Package *

Note:
Consider security aspects when using services via the Internet/Intranet, especially for administrative and development tasks.
For more information about secure handling of administrative tasks, see SAP Notes 1778777 and 1748112.

Now you know how OData services can be consumed by SAPUI5 applications.

To consolidate your knowledge, you can do task 4-6 of the OData exercise.

Odata Function Imports

- Odata originally intended for CRUD operations (*)
- No support of advanced query techniques (aggregation, groupby, ...)
- Function Imports can be used when the intended service does not fit into the CRUD schema

() Newer ODATA Versions (≥ 4) support aggregation via the \$apply query option*

Odata Function Imports

- TA: SEGW
- METHOD /iwbep/ifu_mgw_appl_srv_runtime~execute_action
is responsible for all function imports -> Distinguish between different execution paths by providing different action items in the service request
- Example for action item

```
IF iv_action_name EQ 'StartSimulation'.  
  CALL FUNCTION 'Z_START_SIMULATION'  
  EXPORTING  
    it_parameter = it_parameter.
```
- it_parameter = Table of Key-Value pairs e.g. Airline="AA", User="DEV-300" ...

Check your knowledge

Check your knowledge

- Which statement is true about REST and AJAX?
 - ☐ REST follows the AJAX standard
 - ☐ AJAX follows the REST architectural principles
- How are HTTP, XML and OData related?
- What is the purpose of callback functions?
- Which programming language is used to consume an OData service?
 - ☐ HTML5
 - ☐ ABAP
 - ☐ JavaScript

Solution

Solution

- Which statement is true about REST and AJAX?
 - ☐ REST follows the AJAX standard
 - ☒ AJAX follows the REST architectural principles
- How are HTTP, XML and OData related?

See section *Using the OData protocol*
- What is the purpose of callback functions?

See section *Using the OData protocol*
- Which programming language is used to consume an OData service?
 - ☐ HTML5
 - ☐ ABAP
 - ☒ JavaScript

© 2018 SAP SE or an SAP affiliate company. All rights reserved.

No part of this publication may be reproduced or transmitted in any form or for any purpose without the express permission of SAP SE or an SAP affiliate company.

SAP and other SAP products and services mentioned herein as well as their respective logos are trademarks or registered trademarks of SAP SE (or an SAP affiliate company) in Germany and other countries. Please see <http://global12.sap.com/corporate-en/legal/copyright/index.epx> for additional trademark information and notices.

Some software products marketed by SAP SE and its distributors contain proprietary software components of other software vendors.

National product specifications may vary.

These materials are provided by SAP SE or an SAP affiliate company for informational purposes only, without representation or warranty of any kind, and SAP SE or its affiliated companies shall not be liable for errors or omissions with respect to the materials. The only warranties for SAP SE or SAP affiliate company products and services are those that are set forth in the express warranty statements accompanying such products and services, if any. Nothing herein should be construed as constituting an additional warranty.

In particular, SAP SE or its affiliated companies have no obligation to pursue any course of business outlined in this document or any related presentation, or to develop or release any functionality mentioned therein. This document, or any related presentation, and SAP SE's or its affiliated companies' strategy and possible future developments, products, and/or platform directions and functionality are all subject to change and may be changed by SAP SE or its affiliated companies at any time for any reason without notice. The information in this document is not a commitment, promise, or legal obligation to deliver any material, code, or functionality. All forward-looking statements are subject to various risks and uncertainties that could cause actual results to differ materially from expectations. Readers are cautioned not to place undue reliance on these forward-looking statements, which speak only as of their dates, and they should not be relied upon in making purchasing decisions.